"La Ciencia sin Moral es Vana"

Universidad Católica de El Salvador

Facultad de Ingeniería y Arquitectura

Ingeniería en Desarrollo de Software

Sistema de Gestión de Citas Médicas para Clínica

Primer Avance (20 %)

Cátedra:
Programación de Base de Datos

Catedrático:
Ing. José Ernesto Erazo Linares

Integrantes:
Brian Steven Beltrán Reina
Edwin Geovanni Guerra Cruz
Edgar Alessandro López Martinez

20 de febrero de 2026

Índice

1. Alcance del Proyecto a Realizar **2**
1.1. Funcionalidades Principales 2
1.2. Problemática que Resuelve 2

2. Viabilidad del Proyecto **2**
2.1. Viabilidad Técnica 2
2.2. Impacto en el Entorno Informático 3

3. Diagramas de Arquitectura de Software **3**
3.1. Diagrama de Casos de Uso 3
3.2. Diagrama de Actividades 4
3.3. Diagramas de Contexto (DFD) 6
3.3.1. Diagrama de Contexto (Nivel 0) 6
3.3.2. Diagrama de Nivel 1 6
3.3.3. Diagrama de Nivel 2 6
3.4. Arquitectura de la Aplicación 7
3.5. Primer Diagrama Entidad-Relación (DER) 7

4. Estimación y Costos (Método COCOMO) **8**
4.1. Cálculo de Esfuerzo (E) 9
4.2. Cálculo de Tiempo Real (T) 9
4.3. Estimación de Costo Total del Proyecto 9

5. Metodologías Ágiles **9**
5.1. Seguimiento del Desarrollo 10
5.2. Planificación de Sprints Iniciales 10

6. Documentación Adicional y Funcionalidades Innovadoras **10**
6.1. Funcionalidad Innovadora: Analítica Predictiva 10
6.2. Documentación de API 10

7. Prototipos del Sistema **10**
7.1. Pantalla de Inicio de Sesión 11
7.2. Dashboard Principal con Estadisticas 11
7.3. Interfaz de registro de citas 11

1. Alcance del Proyecto a Realizar

El proyecto consiste en el desarrollo de una plataforma web integral para la gestión clínica, diseñada para optimizar el flujo de trabajo operativo y potenciar la toma de decisiones mediante inteligencia artificial.

1.1. Funcionalidades Principales

El sistema abarcará los siguientes módulos funcionales:

- **Gestión de Usuarios y Roles:** Administración de perfiles para doctores, pacientes y personal administrativo.
- **Gestión de Citas Médicas:** Programación, reprogramación y cancelación de citas, permitiendo visualizar la disponibilidad de los doctores en tiempo real.
- **Expediente Clínico Electrónico:** Registro de consultas, signos vitales (peso, talla, presión arterial), diagnósticos (CIE-10) y recetas médicas.
- **Farmacia e Inventario:** Control de stock de medicamentos, alertas de stock mínimo y vinculación directa con las recetas emitidas.
- **Módulo de Inteligencia Artificial (Analytics):** Integración con un microservicio en Python para analizar datos históricos y generar predicciones sobre ausentismo de pacientes y riesgos de salud basados en signos vitales.

1.2. Problemática que Resuelve

Actualmente, muchas clínicas enfrentan problemas como la pérdida de expedientes físicos, la ineficiencia en la gestión de inventarios y una alta tasa de ausentismo en las citas ("No-Show").

Nuestra base de datos y sistema resuelven esto centralizando la información, garantizando la integridad de los datos médicos y proporcionando herramientas predictivas que permiten a la clínica tomar medidas preventivas (ej. recordatorios personalizados) para reducir pérdidas económicas y mejorar la atención al paciente.

2. Viabilidad del Proyecto

2.1. Viabilidad Técnica

El proyecto es altamente viable tecnológicamente debido al uso de un stack moderno y robusto:

- **Backend:** Java con Spring Boot, estándar en la industria para sistemas empresariales seguros y escalables.
- **Base de Datos:** PostgreSQL, un motor relacional potente capaz de manejar transacciones complejas y grandes volúmenes de datos.
- **Machine Learning:** Python (Pandas, Scikit-learn), líder indiscutible en ciencia de datos.

- **Infraestructura:** Uso de Docker para la contenerización, asegurando que el despliegue sea consistente en cualquier entorno.

2.2. Impacto en el Entorno Informático

Positivo: La adopción de una arquitectura de microservicios (separando la lógica de negocio en Java y el análisis en Python) fomenta la mantenibilidad y escalabilidad. Además, el uso de estándares abiertos reduce la dependencia de licencias costosas.

Negativo/Riesgos: La curva de aprendizaje para integrar dos stacks diferentes (Java y Python) y la gestión de la infraestructura en contenedores requiere un nivel técnico avanzado, lo cual se mitiga con una buena documentación y prácticas DevOps.

3. Diagramas de Arquitectura de Software

En esta sección se detallan los diagramas que modelan el comportamiento y la estructura del sistema.

3.1. Diagrama de Casos de Uso

Muestra las interacciones entre los actores (Paciente, Doctor, Admin) y el sistema.

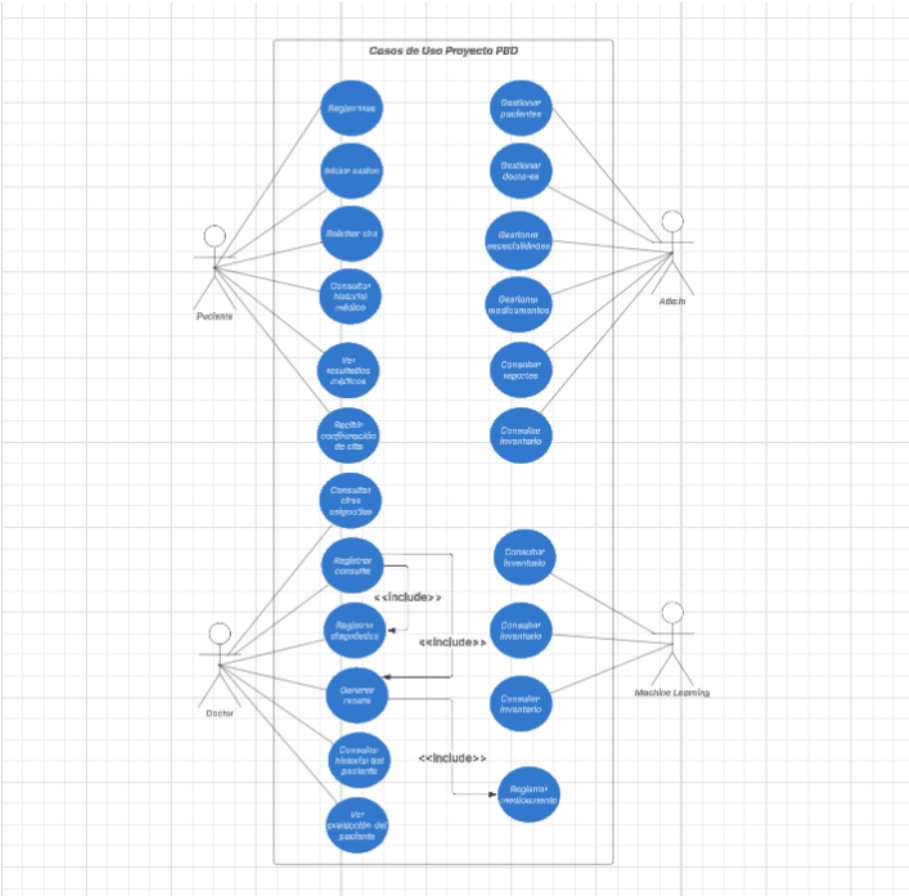
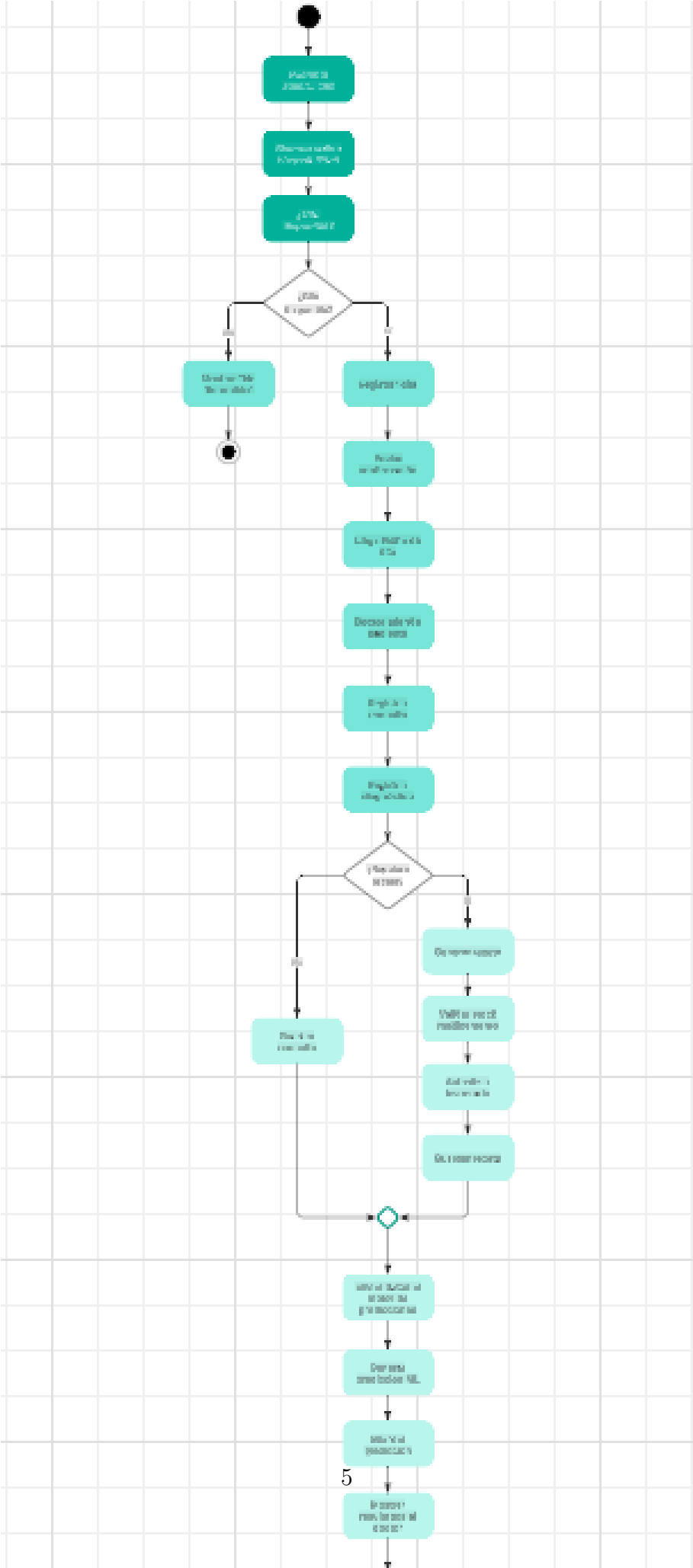


Figura 1: Diagrama de Casos de Uso General

3.2. Diagrama de Actividades

Flujo del proceso principal: Agendamiento y realización de una consulta médica.



3.3. Diagramas de Contexto (DFD)

3.3.1. Diagrama de Contexto (Nivel 0)

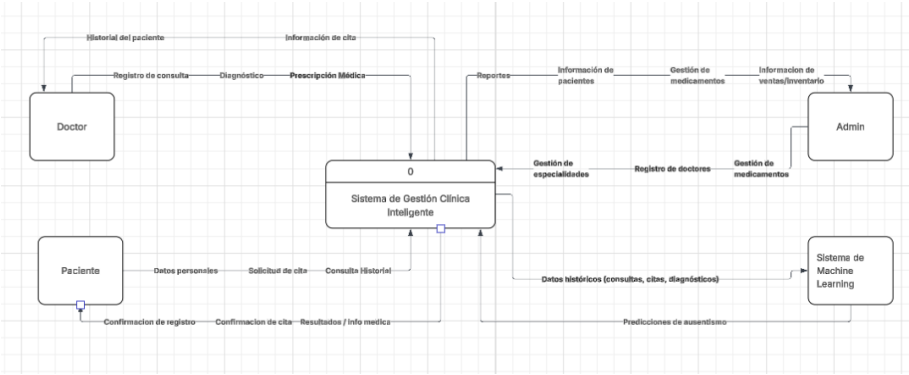


Figura 3: Diagrama de Flujo de Datos - Nivel 0

3.3.2. Diagrama de Nivel 1

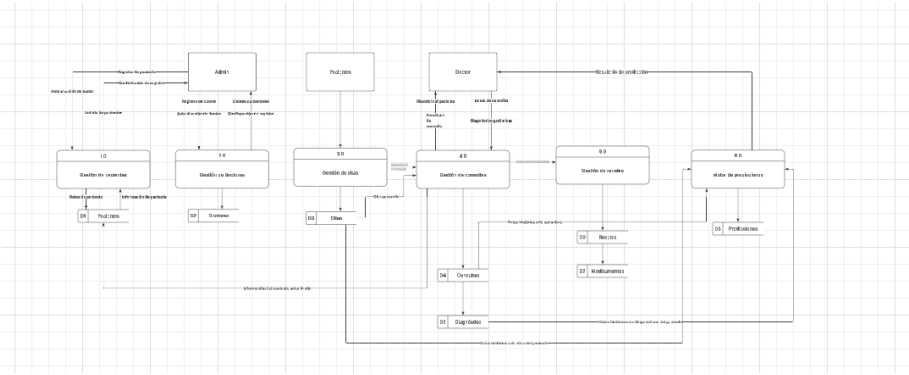


Figura 4: Diagrama de Flujo de Datos - Nivel 1

3.3.3. Diagrama de Nivel 2

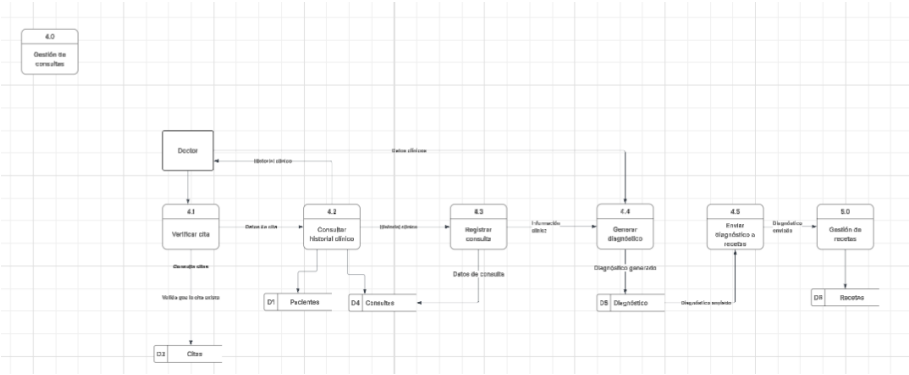


Figura 5: Diagrama de Flujo de Datos - Nivel 2 (Proceso Específico)

3.4. Arquitectura de la Aplicación

Arquitectura híbrida mostrando la comunicación entre Spring Boot, el contenedor de PostgreSQL y el servicio de Machine Learning en Python.

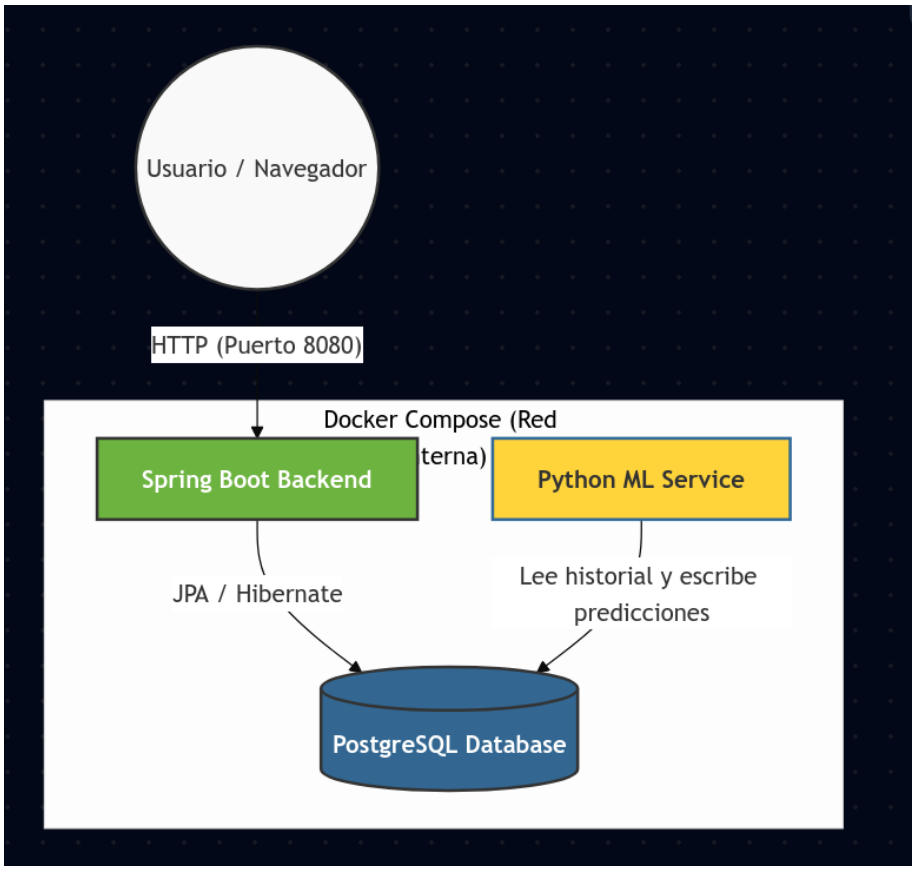


Figura 6: Arquitectura de Microservicios y Contenedores

3.5. Primer Diagrama Entidad-Relación (DER)

Modelo lógico de la base de datos PostgreSQL, incluyendo tablas clave como Pacientes, Doctores, Citas y Predicciones.

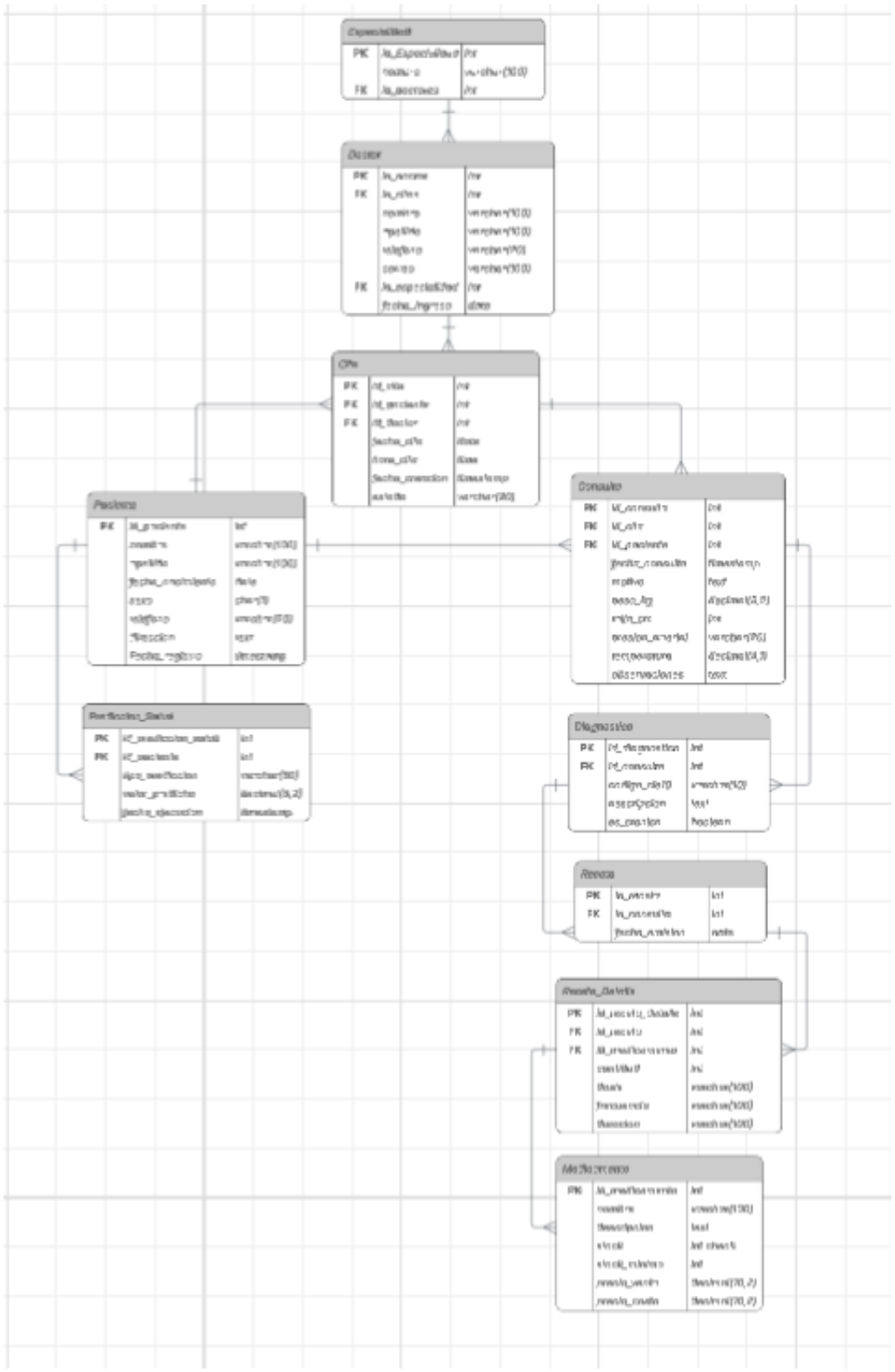


Figura 7: Diagrama Entidad-Relación (Generado desde PostgreSQL)

4. Estimación y Costos (Método COCOMO)

Para la estimación de costos y esfuerzo se ha utilizado el modelo **COCOMO Básico** en modo *Orgánico*, ajustado al equipo de desarrollo conformado por 3 inte-

grantes.

Variables del modelo:

- **KLOC** (Miles de líneas de código estimadas): 4.5 KLOC
- **Equipo de Desarrollo:** 3 Personas (Brian Beltrán, Edwin Guerra, Edgar López)
- **Salario Mensual por Desarrollador:** $800.00 USD (Promedio Junior)
- $a = 2{,}4$, $b = 1{,}05$ (Coeficientes para modo Orgánico)

4.1. Cálculo de Esfuerzo (E)

El esfuerzo representa la cantidad de trabajo requerido en meses/persona.

$$E = a \times (KLOC)^b = 2{,}4 \times (4{,}5)^{1{,}05} \approx 11{,}64 \text{ Personas-Mes}$$

4.2. Cálculo de Tiempo Real (T)

Considerando que el equipo está compuesto por **3 integrantes** trabajando en paralelo, dividimos el esfuerzo total entre el número de desarrolladores:

$$T_{real} = \frac{E}{\text{Nº Desarrolladores}} = \frac{11{,}64}{3} \approx 3{,}88 \text{ Meses}$$

Nota: Esto se alinea con la duración aproximada de un ciclo académico (4 meses).

4.3. Estimación de Costo Total del Proyecto

El costo se calcula multiplicando el esfuerzo total por el salario mensual estimado.

$$Costo = E \times \text{Salario} = 11{,}64 \times \$800{,}00$$

$$\textbf{Costo Total} = \textbf{\$9,312.00 USD}$$

Concepto	**Estimación**
Líneas de Código (KLOC)	4.5
Esfuerzo Total	11.64 Personas-Mes
Equipo de Trabajo	3 Desarrolladores
Tiempo de Desarrollo	**3.88 Meses**
Salario Base (Mes)	$800.00
Costo Total del Software	**$9,312.00 USD**

Cuadro 1: Resumen de Estimación Económica (3 Integrantes)

5. Metodologías Ágiles

El desarrollo se rige bajo el marco de trabajo **Scrum**, permitiendo entregas incrementales y adaptación a cambios.

5.1. Seguimiento del Desarrollo

- **Sprints:** Ciclos de desarrollo de 2 semanas.
- **Herramientas:** Uso de Trello/Jira para el tablero Kanban (Backlog, En Progreso, Terminado) y GitHub para el control de versiones.
- **Daily Meetings:** Reuniones cortas de sincronización para identificar bloqueos.

5.2. Planificación de Sprints Iniciales

- **Sprint 0:** Configuración del entorno Docker, definición de la Base de Datos PostgreSQL y diseño de arquitectura.
- **Sprint 1:** Desarrollo del Backend (Spring Boot) para CRUD de Pacientes y Doctores.
- **Sprint 2:** Implementación del módulo de Citas y conexión con el servicio de ML en Python.

6. Documentación Adicional y Funcionalidades Innovadoras

6.1. Funcionalidad Innovadora: Analítica Predictiva

A diferencia de los sistemas de gestión clínica tradicionales, nuestra base de datos incluye una estructura dedicada (`predicciones_salud`) alimentada por algoritmos de Machine Learning.

- **Predicción de Ausentismo:** Analiza la fecha de creación de la cita vs. la fecha real para predecir la probabilidad de que un paciente no asista, permitiendo a la clínica confirmar con antelación.
- **Alertas de Salud:** Monitoreo de tendencias en signos vitales (presión arterial, peso) para sugerir intervenciones preventivas antes de un diagnóstico crítico.

6.2. Documentación de API

Se utiliza **OpenAPI (Swagger)** para documentar automáticamente los endpoints REST generados por Spring Boot, facilitando la integración con el Frontend y aplicaciones móviles futuras.

7. Prototipos del Sistema

A continuación, se presentan los prototipos funcionales de las pantallas principales del sistema.

7.1. Pantalla de Inicio de Sesión

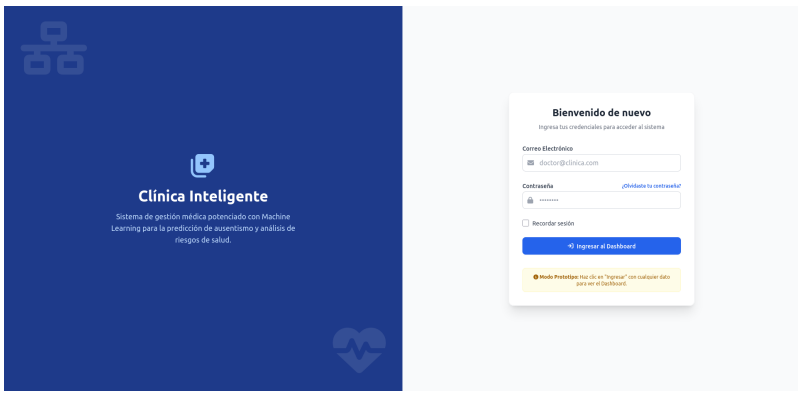


Figura 8: Prototipo de Acceso

7.2. Dashboard Principal con Estadisticas

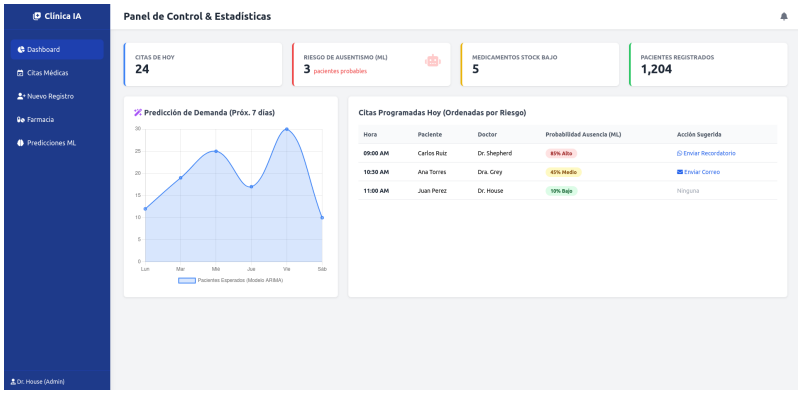


Figura 9: Panel Inicial con datos Estadisticos

7.3. Interfaz de registro de citas

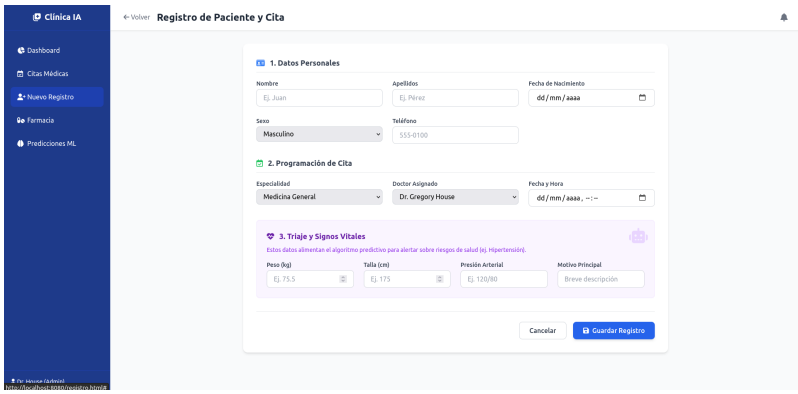


Figura 10: Registro de citas y asignaciones